

Artifact Registry 深入解析

來源：<https://codelabs.developers.google.com/artifact-registry-deepdive?hl=zh-tw>

步驟數：9

目錄

1. [總覽](#)
2. [設定和需求](#)
3. [推送容器映像檔](#)
4. [檢查容器映像檔](#)
5. [虛擬和遠端存放區](#)
6. [部署至 Cloud Run](#)
7. [強化供應鏈安全](#)
8. [管理 Java Maven 語言套件](#)
9. [恭喜！](#)

步驟 1 · 約 0 分鐘

1. 總覽

[Artifact Registry](#) 是全代管的套件管理工具，可提供統一的工具來管理 OCI 容器映像檔和語言套件 (例如 Maven 和 npm)。

Artifact Registry 與 Google Cloud 的各種服務完全整合，例如：

- Cloud Build 可直接將映像檔構件上傳至 Artifact Registry。
- Cloud Deploy 可將 Artifact Registry 映像檔直接部署至各種執行階段環境。
- 為 Cloud Run 和 GKE 等容器執行階段提供映像檔，並啟用映像檔串流等進階效能最佳化功能。
- Artifact Registry 可做為 Artifact Analysis 的偵測點，持續監控已知安全漏洞。
- Cloud IAM 可精細控管構件存取權和權限。

本實驗室將以實作教學課程的形式，逐步說明這些功能。

學習目標

這個實驗室的學習目標為何？

- 為容器和語言套件建立不同的存放區
 - 使用 Artifact Registry 建立及使用容器映像檔
 - 使用 Artifact Registry 分析構件的安全性狀態和內容
 - 設定及使用 Artifact Registry 取得 Java Maven 依附元件
-

步驟 2 · 約 5 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The image shows two screenshots from the Google Cloud console. The top screenshot shows the 'Select a project' dropdown menu in the top navigation bar, with a blue arrow pointing to it. The bottom screenshot shows the 'New Project' form. The 'Project name' field contains 'My Project 13475'. The 'Project ID' field contains 'inbound-analogy-389614', which is highlighted with a blue box. The 'Location' field is empty, and the 'BROWSE' button is visible. The 'CREATE' and 'CANCEL' buttons are at the bottom.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用戶可參加[價值\\$300 美元的免費試用計畫](#)。

設定 gcloud

在 Cloud Shell 設定專案 ID 和專案編號，分別儲存為 `PROJECT_ID` 和 `PROJECT_NUMBER` 變數。

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
```

啟用 Google 服務

```
gcloud services enable \
  cloudfirestore.googleapis.com \
  run.googleapis.com \
  artifactregistry.googleapis.com \
  containerregistry.googleapis.com \
  containerscanning.googleapis.com \
  binaryauthorization.googleapis.com \
  cloudbuild.googleapis.com
```

取得原始碼

本實驗室的原始碼位於 GitHub 的 GoogleCloudPlatform 機構。使用下列指令複製這個存放區。

```
git clone https://github.com/GoogleCloudPlatform/java-docs-samples
```

步驟 3 · 約 10 分鐘

3. 推送容器映像檔

在 Artifact Registry 建立 Docker 存放區

如前所述，Artifact Registry 支援不同的存放區格式，可讓您管理容器映像檔和語言套件。規格中定義了與不同類型構件存放區的互動，並廣泛採用標準。舉例來說，Maven 依附元件和 Node 依附元件的要求就不一樣。

為支援特定構件 API 規格，Artifact Registry 需要在對應的存放區類型中管理構件。建立新存放區時，請傳入 `--repository-format` 旗標，指出存放區類型。

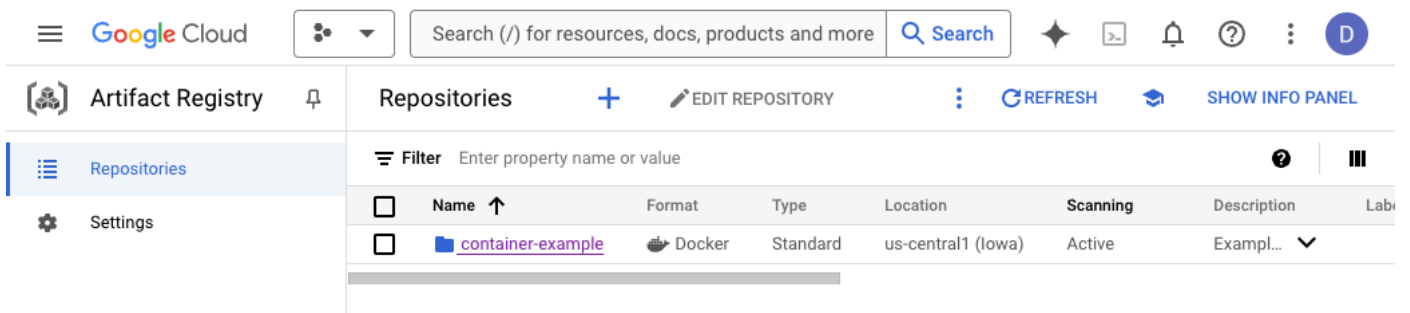
請謹慎選取存放區類型，因為存放區建立後即無法變更。

如要建立第一個 Docker 映像檔存放區，請在 Cloud Shell 中執行下列指令：

```
gcloud artifacts repositories create container-example --repository-format=docker \
--location=us-central1 --description="Example Docker repository"
```

如果出現 Cloud Shell 授權提示，請點選「授權」。

前往 Google Cloud 控制台的「Artifact Registry」-「Repositories」，就會看到新建立的 Docker 存放區 `container-example`。點選後，會發現存放區目前是空的。



設定 Docker 對 Artifact Registry 的驗證機制

連線至 Artifact Registry 時，必須提供憑證才能授予存取權。Docker 可設定為直接使用 gcloud 憑證，不必另外設定憑證。

在 Cloud Shell 執行下列指令，將 Docker 設為使用 Google Cloud CLI，驗證對位在 `us-central1` 區域的 Artifact Registry 傳送的要求：

```
gcloud auth configure-docker us-central1-docker.pkg.dev
```

系統會提示您確認是否要變更 Cloud Shell Docker 設定，請按下 Enter 鍵確認。

探索範例應用程式

您在先前的步驟中複製的 Git 存放區提供範例應用程式。切換至 `java` 目錄並查看應用程式程式碼。

```
cd java-docs-samples/run/helloworld/  
ls
```

這個資料夾包含一個範例 Java 應用程式，可呈現簡單的網頁。除了與本實驗室無關的各種檔案外，還包含 `src` 資料夾中的原始碼，以及我們將用來建構容器映像檔的 Dockerfile。

建構容器映像檔

如要將容器映像檔儲存至 Artifact Registry，必須先建立一個。

執行下列指令，建構容器映像檔並加上完整 Artifact Registry 路徑的標記：

```
docker build -t us-central1-docker.pkg.dev/$PROJECT_ID/container-example/java-hello-  
world:tag1 .
```

將容器映像檔推送至 Artifact Registry：

執行下列指令，將容器映像檔推送至先前建立的存放區：

```
docker push us-central1-docker.pkg.dev/$PROJECT_ID/container-example/java-hello-world:tag1
```

在 Artifact Registry 查看映像檔

前往 Google [Cloud 控制台 - Artifact Registry - 存放區](#)。開啟 `container-example` 存放區，確認是否有 `java-hello-world` 映像檔。

Repository details

Format	Docker
Type	Standard

▼ SHOW MORE

≡ Filter Enter property name or value

☐	Name ↑	Connection	Created	Updated
☐	 java-hello-world	—	1 minute ago	1 minute ago

按一下圖片，注意標有 `tag1` 的圖片。由於我們透過 `containerscanning.googleapis.com` 服務啟用映像檔的自動掃描功能，因此您會看到安全漏洞掃描正在執行或已完成。完成後，您會看到這個映像檔修訂版本偵測到的安全漏洞數量。我們將在下一節探討安全漏洞和其他構件洞察資料。

☒ Hide OCI alternative artifacts

 **Filter** Enter property name or value

☐	Name	Description	Tags 	Created	Updated 	Vulnerabilities	
☐	 71eebbf04bf7		tag1	1 minute ago	1 minute ago	 12	

4. 檢查容器映像檔

現在您已將第一張圖片推送至 `container-example` 存放區，我們可以進一步瞭解。在「版本」分頁中，按一下剛建立的版本。如要顯示更詳細的圖片：

←

71eebbf04bf7d

DELETE

SETUP INSTRUCTIONS

DEPLOY

REFRESH

...

>

sha256:71eebbf04bf7d1d023e5de5e18f786ea3b8b6411bf64c8def3301c71baca0518

OVERVIEW

DEPENDENCIES

VULNERABILITIES

PULL

MANIFEST

FILES

ATTACHMENTS

Format

Docker

Media type

application/vnd.docker.distribution.manifest.v2+json

Project

my-project

Location

us-central1 (Iowa)

Repository

container-example

Image

java-hello-world

Digest

sha256:71eebbf04bf7d1d023e5de5e18f786ea3b8b6411bf64c8def3301c71baca0518

Virtual size

74.2 MB

Built

29 Nov 2024, 09:51:23

Created

29 Nov 2024, 09:52:27

Updated

29 Nov 2024, 09:52:27

Tags

tag1

Build

—

Subject digest

—

Artifact type

—

上方的複製按鈕特別實用，因為它提供簡單的方式來存取該映像檔版本的完整映像檔 URI，包括 SHA 雜湊值。這個 URI 之後可用於提取特定映像檔版本，或做為 Kubernetes Deployment 或 Cloud Run Service 中的映像檔參照。如要測試圖片 URI，請在 Cloud Shell 中執行下列指令：

```
docker pull $IMAGE_URI
```

瞭解依附元件

移至頂端的「依附元件」分頁，即可查看映像檔中偵測到的所有依附元件。請注意，這份清單列出語言和作業系統層級的依附元件。您也可以查看附加至各個依附元件的軟體授權。

us-central1-docker.pkg.dev > my-project > container-example > java-hello-world > sha256:71eebbf04bf7d1d023e5de5e18f786ea3b8b6411bf64c8def3301c71baca0518

OVERVIEWDEPENDENCIESVULNERABILITIESPULLMANIFESTFILESATTACHMENTS

Dependencies are part of SBOMs. Learn how to generate and store SBOMs to populate this page. Learn more



Filter Filter dependencies

Name	Version	Package type	Licence
alpine-baselayout	3.6.5-r0	OS	GPL-2.0-only
alpine-baselayout-data	3.6.5-r0	OS	GPL-2.0-only
alpine-keys	2.4-r1	OS	MIT
apk-tools	2.14.4-r0	OS	GPL-2.0-only
brothi-libs	1.1.0-r2	OS	MIT
busybox	1.36.1-r29	OS	GPL-2.0-only
busybox-binsh	1.36.1-r29	OS	GPL-2.0-only
ca-certificates	20240705-r0	OS	MIT AND MPL-2.0
ca-certificates-bundle	20240705-r0	OS	MIT AND MPL-2.0
ch.qos.logback.logback-classic	1.4.14	Maven	EPL-1.0 AND non-standard
ch.qos.logback.logback-core	1.4.14	Maven	EPL-1.0 AND non-standard

仔細查看後，您會發現 SBOM 資訊尚未填入。如要填入構件的 SOM，請在 Cloud Shell 中執行下列指令，並使用可從頂端麵包屑列複製的完整映像檔 URI。

```
gcloud artifacts sbom export --uri $IMAGE_URI
```

重新整理頁面後，您會看到系統自動產生的 SBOM 連結，該連結會儲存在 Cloud Storage 中。如果您依賴映像檔的 SBOM，不妨為構件自動產生 SBOM，並將產生作業納入 CI/CD 管道。

探索映像檔安全漏洞

按一下檢視畫面頂端的「安全漏洞」分頁，即可查看映像檔中偵測到的所有安全漏洞。除了頂端的安全漏洞摘要，您還可以在底部的表格中查看完整清單。每列都會連結至 CVE 公告，指出嚴重程度和來源套件。如果安全漏洞有可用的修正程式，系統也會提供更新依附元件的指示，以修正安全漏洞。

us-central1-docker.pkg.dev > my-project > container-example > java-hello-world > sha256:71eebbf04bf7d1d023e5de5e18f786ea3b8b6411bf64c8def3301c71baca0518

OVERVIEWDEPENDENCIESVULNERABILITIESPULLMANIFESTFILESATTACHMENTS

Scan results

Effective severity based on factors such as exploitability, scope, impact and maturity of the vulnerability.

Scan details

Scanning enablement: Scanning active

Package types scanned: Maven, Npm, OS, Python, Rubygems, Composer, Rust, Nuget, Go

Total vulnerabilities: 12

Fixes available: 12

No fix available

VEX status: PREVIEW

Affected	—
Fixed	—
Not affected	—
Under investigation	—
Unspecified	12

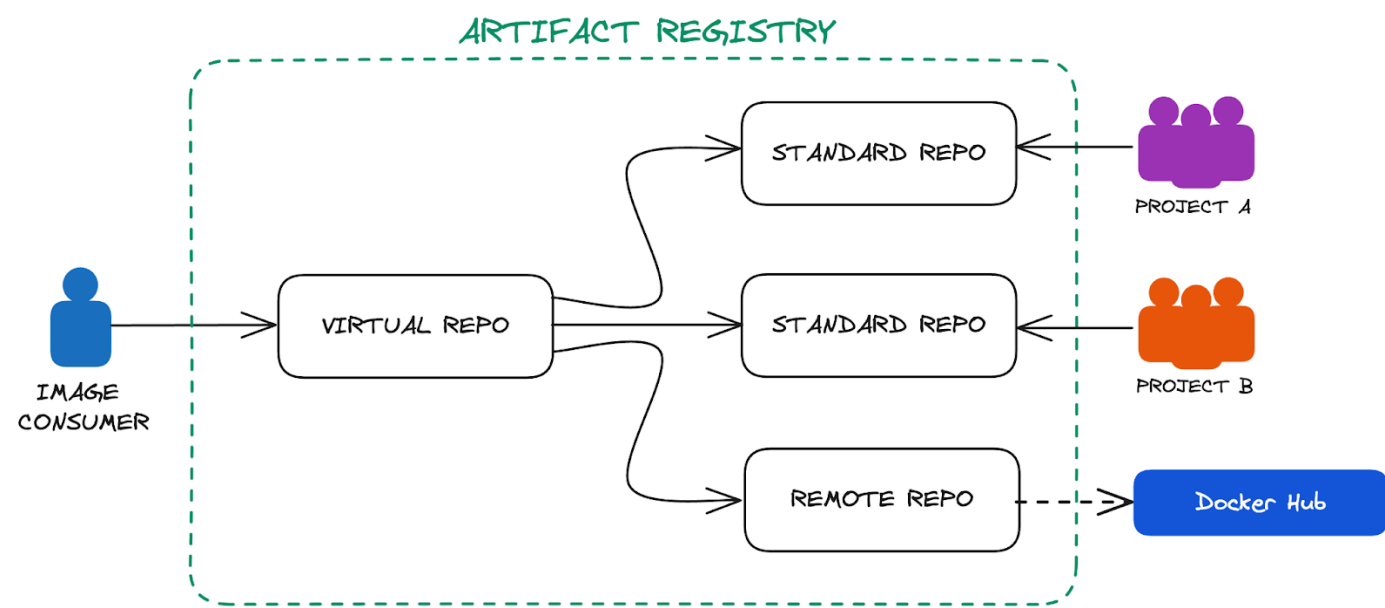
SHOW MORE

Filter Filter vulnerabilities

Name	Effective severity	CVSS	Fix available	VEX status	Package	Package type
CVE-2024-22262	High	0	Yes	Unspecified	org.springframework:spring-web	Maven VIEW FIX
CVE-2024-38816	High	0	Yes	Unspecified	org.springframework:spring-webmvc	Maven VIEW FIX
CVE-2024-22259	High	0	Yes	Unspecified	org.springframework:spring-web	Maven VIEW FIX
CVE-2024-22243	High	0	Yes	Unspecified	org.springframework:spring-web	Maven VIEW FIX
CVE-2024-34750	High	0	Yes	Unspecified	org.apache.tomcat.embed:tomcat-embed-core	Maven VIEW FIX

5. 虛擬和遠端存放區

在上一節中，我們使用單一映像檔存放區推送及提取映像檔。這項功能非常適合小規模用途，但對於需要自主管理存放區的不同團隊來說，尤其是大型機構，可能會面臨挑戰。團隊或業務單位通常會擁有自己的映像檔存放區，並具備專屬權限和設定。為簡化這些存放區中映像檔的使用方式，並避免消費者受到基礎組織結構影響，Artifact Registry 提供虛擬存放區，可匯總多個基礎存放區的資源。潛在架構可能如下所示：



Docker Hub 的遠端存放區

Docker Hub 是熱門的公開映像檔登錄檔，內含許多開放原始碼容器映像檔。直接使用公開存放區很簡單，但在正式環境中會遇到許多挑戰，而 Artifact Registry 中的遠端存放區可以解決這些問題。

遠端存放區可將要求 Proxy 至上游登錄檔，並在過程中快取映像檔。這不僅能縮短圖片的下載時間，還能移除對外部服務正常運作時間的依附性，讓您套用與自有圖片相同的安全性和存取政策。

如要為 Docker Hub 建立遠端存放區，請在 Cloud Shell 中執行下列指令：

```
gcloud artifacts repositories create dockerhub \
  --repository-format=docker \
  --mode=remote-repository \
  --remote-docker-repo=docker-hub \
  --location=us-central1 \
  --description="Example Remote Repo for Docker Hub"
```

現在，Artifact Registry 存放區清單中應該會顯示額外存放區：

Repositories								
+ EDIT REPOSITORY DELETE REFRESH SHOW INFO PANEL								
Filter Enter property name or value								
☐	Name ↑	Format	Type	Location	Scanning	Description	Labels	Version policy
☐	container-example	Docker	Standard	us-central1 (Iowa)	Active	Exempl...	▼	—
☐	dockerhub	Docker	Remote	us-central1 (Iowa)	Active	Exempl...	▼	—

如要測試遠端存放區是否能將要求 Proxy 至遠端存放區，請在 Cloud Shell 中執行下列指令，提取 nginx 映像檔：

```
docker pull us-central1-docker.pkg.dev/$PROJECT_ID/dockerhub/nginx:stable-alpine
```

提取成功後，您也可以到 Cloud 控制台中查看存放區，並確認快取的 Nginx 映像檔現在提供的依附元件和安全漏洞報告，與您自行建構的映像檔相同。

建立虛擬存放區

按照我們目前使用的程序，您可以為每個團隊或商家建立多個存放區，並清楚定義每個存放區的擁有權和 IAM 權限。我們也可以為遠端存放區建立 Proxy，讓您更輕鬆安全地使用第三方映像檔。如果從這些映像檔的消費者角度來看，大量存放區的缺點顯而易見。開發人員如何判斷部署時應使用哪個映像檔存放區？

若要簡化使用程序，並在抽象層後方隱藏基礎存放區，您可以在 Artifact Registry 中建立虛擬存放區，方法是在 Cloud Shell 中執行下列指令：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member serviceAccount:service-$PROJECT_NUMBER@gcp-sa-
artifactregistry.iam.gserviceaccount.com \
  --role roles/artifactregistry.reader

cat <<EOF > /tmp/upstream.json
[{
  "id" : "hello-repo",
  "repository" : "projects/$PROJECT_ID/locations/us-central1/repositories/container-example",
  "priority" : 100
},{
  "id" : "dockerhub",
  "repository" : "projects/$PROJECT_ID/locations/us-central1/repositories/dockerhub",
  "priority" : 101
}]
EOF

gcloud artifacts repositories create all-images \
  --repository-format=docker \
  --mode=virtual-repository \
  --location=us-central1 \
  --upstream-policy-file=/tmp/upstream.json \
  --description="Example Virtual Repo"
```

現在可以從虛擬存放區提取映像檔，不必公開基礎結構。若要確認一切運作正常，可以在 Cloud Shell 中執行下列指令：

```
docker pull us-central1-docker.pkg.dev/$PROJECT_ID/all-images/java-hello-world:tag1

docker pull us-central1-docker.pkg.dev/$PROJECT_ID/all-images/nginx:stable-alpine
```

步驟 6 · 約 5 分鐘

6. 部署至 Cloud Run

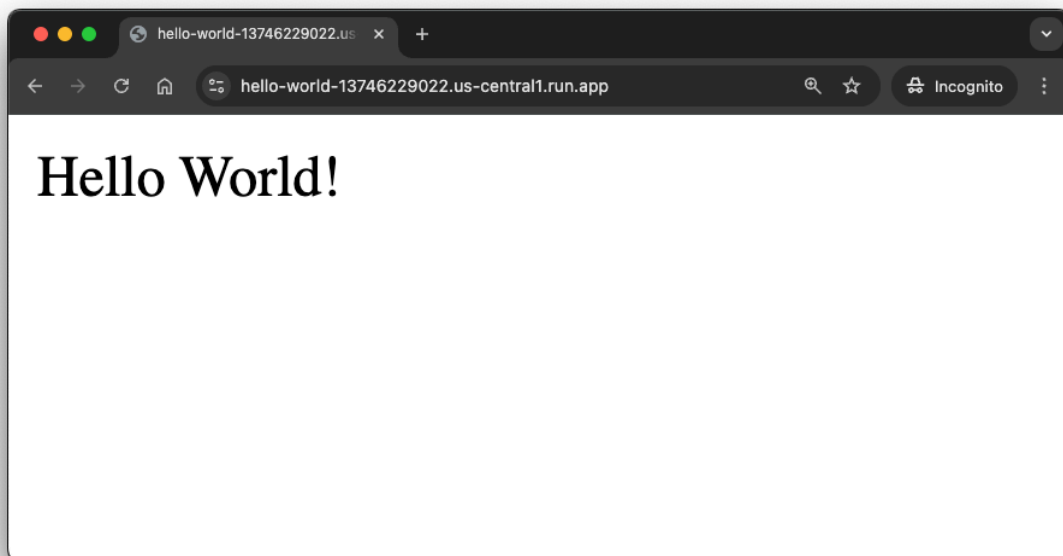
有了適當的存放區和映像檔，我們現在可以開始在部署作業中使用這些項目。為說明範例用途並避免部署額外基礎架構，我們將容器部署至 Cloud Run。最簡單的部署方式是在 Cloud Shell 中執行下列指令：

```
gcloud run deploy hello-world \
  --image us-central1-docker.pkg.dev/$PROJECT_ID/all-images/java-hello-world:tag1 \
  --region us-central1 \
  --allow-unauthenticated
```

部署作業完成後，系統會顯示自動產生的網址，您可透過該網址存取服務。

```
Deploying container to Cloud Run service [hello-world] in project [my-project] region
[us-central1]
OK Deploying... Done.
  OK Creating Revision...
  OK Routing traffic...
  OK Setting IAM Policy...
Done.
Service [hello-world] revision [hello-world-00001-wtc] has been deployed and is
serving 100 percent of traffic.
Service URL: https://hello-world-13746229022.us-central1.run.app
```

在新瀏覽器分頁中開啟該網址，應該會看到「Hello World」訊息，表示函式已成功執行。



7. 強化供應鏈安全

容器映像檔已部署到正式環境，現在或許是強化端對端供應鏈的好時機。在上一節中，我們探討了 Artifact Registry 的容器分析如何深入瞭解映像檔中使用的程式庫和授權。不過，惡意人士仍有可能在供應鏈中將有害內容導入圖片。在本節中，我們將探討如何使用 [SLSA 架構](#)，為建構構件導入驗證，甚至運用構件本身的加密簽章，確保只有受信任的構件可以部署至 Cloud Run 執行階段。

使用 Cloud Build 進行 SLSA 認證

SLSA 架構提供不同層級的供應鏈構件證據。規格和實作方式乍看之下可能令人望而生畏，但使用 Cloud Build 建立 SLSA 認證非常簡單，只要新增 `cloudbuild.yaml` 規格，並將 `requestedVerifyOption` 設為 `VERIFIED` 即可。

對於我們的應用程式，我們可以在 Cloud Shell 中執行下列指令，在 `helloworld` 資料夾中建立 `cloudbuild.yaml` 檔案。

```
cat <<EOF > cloudbuild.yaml
steps:
- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', '\$_IMAGE_URI', '.']
- name: 'gcr.io/cloud-builders/docker'
  args: ['push', '\$_IMAGE_URI']
images:
- '\$_IMAGE_URI'
options:
  requestedVerifyOption: VERIFIED
substitutions:
  _IMAGE_URI: us-central1-docker.pkg.dev/$PROJECT_ID/container-example/java-hello-world:latest
EOF
```

現在，我們要在 Cloud Shell 中執行下列指令，建立新的 Cloud Build 作業，建構新版 Java Hello World 映像檔。

```
gcloud builds submit --substitutions=_IMAGE_URI=us-central1-docker.pkg.dev/$PROJECT_ID/container-example/java-hello-world:cloud-build
```

建構作業完成後，我們可以前往 Google Cloud 控制台的 Cloud Build UI，開啟建構作業，然後查看「建構摘要」下方的「建構構件」，瞭解我們達成的 SLSA 等級。該圖片上有一個按鈕，可查看「安全性深入分析」。按一下即可查看 SLSA 認證，以及我們在推送本機建構作業時，於 Artifact Registry UI 中看到的常見安全漏洞報告。

Software Delivery Shield is a new solution to safeguard artifact integrity across your entire software delivery lifecycle. [Learn more](#) about how it can prevent tampering, improve integrity and secure packages and infrastructure.

 Artifact (java-hello-world)

SLSA build level 3

[What's this?](#)

 Vulnerabilities

Unique vulnerabilities found in scanned containers

Artifacts scanned	java-hello-world
Last scanned	3 minutes ago
Scanning enablement	 Scanning active

 Fixes available

Total	12
-------	--------------------

 No fix available

 Critical	—
 High	—
 Medium	—
 Low	—

 VEX status Preview

Affected	—
Fixed	—
Not affected	—
Under investigation	—
Unspecified	12

[Show more](#)

 Dependencies

No SBOM data found

Build

Logs	d6e35440
Builder	Cloud Build
Provenance	SLSA v0.1 ↗
Completed	3 minutes ago

您也可以在 Cloud Shell 中執行下列指令，擷取映像檔的 SLSA 出處：

```
gcloud artifacts docker images describe \  
"us-central1-docker.pkg.dev/$PROJECT_ID/container-example/java-hello-world:cloud-build" \  
--show-provenance
```

透過二進位授權要求 Cloud Build 來源

有了 Cloud Build 管道，如果能確保部署至正式環境的所有映像檔，都是使用可程式化且可重現的建構環境建構，豈不是美事一樁？

這時，二進位授權就派上用場了。您可以在容器執行階段前方設置守門員，檢查容器映像檔並驗證是否存在可信的認證。如果找不到認證，系統會視設定建立稽核記錄項目，或完全禁止部署作業。

如要將專案的預設 Binary Authorization 設定變更為需要 Cloud Run 簽發的內建驗證，請在 Cloud Shell 中執行下列指令：

```
cat << EOF > /tmp/policy.yaml  
defaultAdmissionRule:  
  enforcementMode: ENFORCED_BLOCK_AND_AUDIT_LOG  
  evaluationMode: REQUIRE_ATTESTATION  
  requireAttestationsBy:  
    - projects/$PROJECT_ID/attestors/built-by-cloud-build  
name: projects/$PROJECT_ID/policy  
EOF  
  
gcloud container binauthz policy import /tmp/policy.yaml
```

您也可以在此新增自己的自訂認證者，但這不在本程式碼研究室的範圍內，因此做為選修的課外作業。

如要在 Cloud Run 服務上強制執行二進位檔授權，請在 Cloud Shell 中執行下列指令：

```
gcloud run services update hello-world \  
--region us-central1 \  
--binary-authorization=default
```

讓我們重新部署本機建構的映像檔，測試二進位授權是否已正確套用


```
gcloud run deploy hello-world \  
  --image us-central1-docker.pkg.dev/$PROJECT_ID/all-images/java-hello-world:tag1 \  
  --region us-central1
```

如預期，您應該會收到錯誤訊息，說明無法部署映像檔的原因，如下所示：

```
Image us-central1-docker.pkg.dev/my-project/all-images/java-hello-world@sha256:71eebbf04bf7d1d023e5de5e18f786ea3b8b6411bf64c8def3301c71baca0518 denied by attestor projects/my-project/attestors/built-by-cloud-build: No attestations found that were valid and signed by a key trusted by the attestor
```

因此，如要將新版本部署至 Cloud Run 服務，我們需要提供使用 Cloud Build 建構的映像檔。

```
gcloud run deploy hello-world \  
  --image us-central1-docker.pkg.dev/$PROJECT_ID/all-images/java-hello-world:cloud-build \  
  --region us-central1
```

這次部署作業應該會成功，並顯示類似下方的成功部署訊息：

```
Deploying container to Cloud Run service [hello-world] in project [my-project] region [us-central1]  
OK Deploying... Done.  
  OK Creating Revision...  
  OK Routing traffic...  
Done.  
Service [hello-world] revision [hello-world-00005-mq4] has been deployed and is serving 100 percent of traffic.  
Service URL: https://hello-world-13746229022.us-central1.run.app
```

步驟 8 · 約 10 分鐘

8. 管理 Java Maven 語言套件

在本節中，您將瞭解如何設定 Artifact Registry Java 存放區並上傳套件，然後在不同應用程式中運用這些套件。

建立 Maven 套件存放區

在 Cloud Shell 執行下列指令，為 Java 構件建立存放區：

```
gcloud artifacts repositories create java-repo \  
  --repository-format=maven \  
  --location=us-central1 \  
  --description="Example Java Maven Repo"
```

如果出現 Cloud Shell 授權提示，請點選「授權」。

前往 Google Cloud 控制台的「Artifact Registry」-「Repositories」，就會看到新建立的 Maven 存放區 `java-repo`。點選後，會發現存放區目前是空的。

設定 Artifact Repository 的驗證機制

使用下列指令，以使用者帳戶憑證更新應用程式預設憑證 (ADC) 的已知位置，這樣 Artifact Registry 憑證輔助程式就能在連線至存放區時，使用這些憑證驗證：

```
gcloud auth login --update-adc
```

設定 Maven 以搭配 Artifact Registry 使用

執行下列指令，顯示要新增至 Java 專案的存放區設定：

```
gcloud artifacts print-settings mvn \  
  --repository=java-repo \  
  --location=us-central1 | tee config.xml
```

在 Cloud Shell 中，從 helloworld 目錄執行下列指令，在 Cloud Shell 編輯器中開啟 pom.xml：

```
cloudshell edit pom.xml
```

將傳回的設定新增至檔案中的對應區段，

更新 **distributionManagement** 區段

```
<distributionManagement>
  <snapshotRepository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/<PROJECT>/java-repo</url>
  </snapshotRepository>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/<PROJECT>/java-repo</url>
  </repository>
</distributionManagement>
```

更新 **repositories** 區段

```
<repositories>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-central1-maven.pkg.dev/<PROJECT>/java-repo</url>
    <releases>
      <enabled>true</enabled>
    </releases>
    <snapshots>
      <enabled>true</enabled>
    </snapshots>
  </repository>
</repositories>
```

更新「build」底下的「extensions」

```
<extensions>
  <extension>
    <groupId>com.google.cloud.artifactregistry</groupId>
    <artifactId>artifactregistry-maven-wagon</artifactId>
    <version>2.1.0</version>
  </extension>
</extensions>
```

您可以參考以下完整檔案的範例。請務必將 <PROJECT> 替換為您的專案 ID。

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example.run</groupId>
  <artifactId>helloworld</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <parent>
    <groupId>com.google.cloud.samples</groupId>
    <artifactId>shared-configuration</artifactId>
    <version>1.2.0</version>
  </parent>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-dependencies</artifactId>
        <version>${spring-boot.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <project.reporting.outputEncoding>UTF-8</project.reporting.outputEncoding>
    <maven.compiler.target>17</maven.compiler.target>
    <maven.compiler.source>17</maven.compiler.source>
    <spring-boot.version>3.2.2</spring-boot.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.junit.vintage</groupId>
      <artifactId>junit-vintage-engine</artifactId>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
```

```

        <scope>test</scope>
    </dependency>
</dependencies>
<!-- [START Artifact Registry Config] -->
<distributionManagement>
    <snapshotRepository>
        <id>artifact-registry</id>
        <url>artifactregistry://us-centrall1-maven.pkg.dev/<PROJECT>/java-repo</url>
    </snapshotRepository>
    <repository>
        <id>artifact-registry</id>
        <url>artifactregistry://us-centrall1-maven.pkg.dev/<PROJECT>/java-repo</url>
    </repository>
</distributionManagement>

<repositories>
    <repository>
        <id>artifact-registry</id>
        <url>artifactregistry://us-centrall1-maven.pkg.dev/<PROJECT>/java-repo</url>
        <releases>
            <enabled>true</enabled>
        </releases>
        <snapshots>
            <enabled>true</enabled>
        </snapshots>
    </repository>
</repositories>
<build>
    <extensions>
        <extension>
            <groupId>com.google.cloud.artifactregistry</groupId>
            <artifactId>artifactregistry-maven-wagon</artifactId>
            <version>2.2.0</version>
        </extension>
    </extensions>
<!-- [END Artifact Registry Config] -->

<plugins>
    <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <version>${spring-boot.version}</version>
        <executions>
            <execution>
                <goals>
                    <goal>repackage</goal>
                </goals>
            </execution>
        </executions>
    </plugin>
    <plugin>
        <groupId>com.google.cloud.tools</groupId>

```

```
<artifactId>jib-maven-plugin</artifactId>
<version>3.4.0</version>
<configuration>
  <to>
    <image>gcr.io/PROJECT_ID/helloworld</image>
  </to>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

將 Java 套件上傳至 Artifact Registry

在 Maven 設定 Artifact Registry 後，即可使用 Artifact Registry 儲存 Java JAR，供貴機構的其他專案使用。

執行下列指令，將 Java 套件上傳至 Artifact Registry：

```
mvn deploy
```

在 Artifact Registry 檢查 Java 套件

前往 [Cloud 控制台 - Artifact Registry - 存放區](#)，然後點按 `java-repo`，確認 `helloworld` 二進位檔構件已存在：

[←](#) Packages for java-repo [DELETE](#) [EDIT REPOSITORY](#) [SETUP INSTRUCTIONS](#) [REFRESH](#)

my-project > us-central1 > java-repo

Repository details

Format	Maven
Type	Standard

[SHOW MORE](#)

Filter Enter property name or value

☐	Name ↑	Created	Updated
☐	com.example.run:helloworld	Just now	Just now

步驟 9 · 約 0 分鐘

9. 恭喜！

恭喜，您已完成本程式碼研究室！

涵蓋內容

- 為容器和語言套件建立存放區
- 使用 Artifact Registry 管理容器映像檔
- 將 Artifact Registry 與 Cloud Code 整合
- 設定 Maven 來使用 Artifact Registry 取得 Java 依附元件

清除所用資源

在 Cloud Shell 中執行下列指令，刪除整個專案

```
gcloud projects delete $PROJECT_ID
```
